

# Wavetable Operator Validation Guide

This guide is for **human listening validation** of the current vivid-wavetable package after the WavetableLayer cutover.

The active production path is now:

PolyVoiceAllocator -> WavetableLayer -> voice\_gain\_audio -> stereo output

WavetableOsc + VoiceMixer remains available for advanced legacy behavior that WavetableLayer intentionally excludes, especially FM/PM/RM/AM oscillator interaction and feedback-style warp. VoiceMixer also remains the normal reduction stage for auxiliary per-voice sources such as SubOsc, AnalogOsc, and NoiseLayer when those layers are mixed beside a WavetableLayer body.

Automated regression checks already cover package manifests, module surfaces, graph load/smoke behavior, renderer correctness, and scalar/optimized backend equivalence. This document is for the listening checks that still need human ears.

## Production Validation Surface

Validate these as the active production path:

- PolyVoiceAllocator
- WavetableLayer
- EnvelopeAu driving WavetableLayer/voice\_gain\_audio
- LayerPad and Layer-based module instruments
- optional character layers such as SubOsc, AnalogOsc, and NoiseLayer when reduced separately before final stereo mixing

Validate these as non-primary surfaces:

- WavetableOsc
- VoiceMixer when it is reducing a legacy wavetable voice or an auxiliary per-voice layer
- GlassInteractionKeys and other interaction patches using mod\_input, FM/PM/RM/AM, or feedback-style warp

## What Lanes Mean

In vivid-wavetable, one note becomes one lane.

- PolyVoiceAllocator turns notes into frequencies, gates, velocities, and lane\_ids.

- WavetableLayer consumes those lanes internally, renders all unison voices, and outputs one stereo signal.
- EnvelopeAu/value -> WavetableLayer/voice\_gain\_audio is the normal per-note amplitude path.
- Extra per-note sources such as SubOsc, AnalogOsc, and NoiseLayer may still need their own reduction before being mixed with the WavetableLayer stereo bus.

Think of it like this:

- before WavetableLayer, note lanes describe playable voices
- inside WavetableLayer, voice rendering and unison summing happen in one production operator
- after WavetableLayer, the signal is normal stereo audio

## Stage 1: Silent Layer Sanity

Build this graph:

- WavetableLayer named osc
- audio\_out named out

Connect:

osc/output -> out/input

Set these params on osc:

- amplitude = 0.25
- wavetable\_source = Builtin
- wavetable\_family = AnalogWarm
- wavetable\_member = Core
- position = 0.35

What you should hear:

- silence

Why this is correct:

- WavetableLayer should not self-start without note lanes
- it needs lane inputs such as frequency, gate, and lane id before it should make sound

If it sounds wrong:

- if you hear a pitched tone or hiss, the production layer is not respecting note-lane input

## Stage 2: Add PolyVoiceAllocator

Add:

- ClockAu named clock
- ChordProgressionAu named chords
- PolyVoiceAllocator named voices

Connect:

```
clock/beat_phase -> chords/beat_phase
chords/notes -> voices/notes_in
chords/velocities -> voices/velocities_in
chords/gates -> voices/gates_in
voices/frequencies -> osc/frequencies
voices/gates -> osc/gates
voices/velocities -> osc/velocities
voices/lane_ids -> osc/lane_ids
```

Recommended params:

- clock/bpm = 96
- voices/max\_voices = 6
- osc/amplitude = 0.20

What you should hear:

- a raw sustained polyphonic wavetable tone
- note changes should follow the chord source

If it sounds wrong:

- silence usually means one of the four voice-lane connections is missing
- stuck or smeared notes usually means lane\_ids is missing or unstable

## Stage 3: Add Per-Note Amplitude

Add:

- EnvelopeAu named amp\_env

Connect:

```
voices/gates -> amp_env/gate
voices/lane_ids -> amp_env/lane_ids
amp_env/value -> osc/voice_gain_audio
```

Recommended params:

- amp\_env/attack = 0.01
- amp\_env/decay = 0.25
- amp\_env/sustain = 0.70
- amp\_env/release = 0.40

What you should hear:

- notes articulate cleanly instead of staying constantly open
- release tails should remain stable during chord changes

If it sounds wrong:

- if notes do not release, confirm `amp_env/value` reaches `osc/voice_gain_audio`
- if voices click or swap envelopes, confirm both `voices/lane_ids -> osc/lane_ids` and `voices/lane_ids -> amp_env/lane_ids`

## Stage 4: Validate Wavetable Motion

Use the same graph and add one modulation source at a time.

Position motion:

`LfoAu/value -> osc/position_mod_audio`

Warp motion:

`LfoAu/value -> osc/warp_mod_audio`

Recommended checks:

- changing `wavetable_family` and `wavetable_member` should clearly change source color
- changing `position` should change timbre, not note pitch
- `warp_mode` and `warp_amount` should change edge or motion while staying bounded
- enabling drift should thicken motion without runaway level changes

If it sounds wrong:

- if modulation changes pitch instead of tone, check that it is patched to `position_mod_audio` or `warp_mod_audio`, not `pitch_mod_audio`
- if a warp mode is expected to be FM feedback, that is a legacy `WavetableOsc` check, not a `WavetableLayer` check

## Stage 5: Validate Character Layers

For extra sources, keep `WavetableLayer` as the main wavetable body and reduce extra per-note sources separately before final stereo mixing.

Typical pattern:

```
voices/... -> WavetableLayer/...
amp_env/value -> WavetableLayer/voice_gain_audio
voices/... -> SubOsc/...
SubOsc/output -> VoiceMixer/input
amp_env/value -> VoiceMixer/amp_env_audio
```

WavetableLayer/output -> Mixer/input\_1  
VoiceMixer/output -> Mixer/input\_2  
Mixer/output -> audio\_out/input

What you should hear:

- the WavetableLayer body remains the main stereo source
- sub, analog, or noise layers support the body without forcing the WavetableLayer body back through a per-voice mixer path

If it sounds wrong:

- if the patch collapses or lanes feel confused, check that WavetableLayer/output is not routed into VoiceMixer/input
- if an auxiliary per-note source is silent, confirm it still has its own VoiceMixer and envelope input

## Stage 6: Reference Instrument Checks

Load and listen to the active production references:

- graphs/presets/layer\_pad\_instrument.json
- graphs/core/wavetable\_layer\_pad\_demo.json
- migrated non-interaction instruments such as bright\_pluck\_instrument.json, rooted\_sub\_bass\_instrument.json, metallic\_hollow\_lead\_instrument.json, and motion\_texture\_instrument.json

Expected result:

- all should produce non-silent stereo output
- LayerPad should remain the clearest recommended production pad path
- migrated instruments may differ slightly from the old WavetableOsc + VoiceMixer versions, but should preserve their role, rough loudness, and musical identity

## Advanced Legacy Checks

Use this section only when validating features that are intentionally outside WavetableLayer v1.

Retained advanced legacy content includes:

- modules/glass\_interaction\_keys.vivid-module.json
- graphs/presets/glass\_interaction\_instrument.json
- graphs/presets/controlled\_metallic\_lead.json
- graphs/presets/fm\_glass\_keys.json
- graphs/presets/growl\_crossmod\_bass.json
- graphs/presets/hybrid\_motion\_arp.json
- graphs/presets/spectral\_interaction\_texture.json
- graphs/presets/orbit\_drone.json

Validate these only for the advanced interaction or feedback behavior they preserve:

- WavetableOsc/mod\_input drives the intended carrier
- FM/PM/RM/AM depth changes are audible and controllable
- feedback-style warp remains bounded and musically useful
- these patches are labeled legacy/advanced and are not presented as the recommended production path

## Final Checklist

- WavetableLayer stays silent without note lanes.
- PolyVoiceAllocator lanes make WavetableLayer produce a playable stereo voice.
- EnvelopeAu/value -> voice\_gain\_audio gives per-note articulation and release.
- Position and warp modulation change timbre rather than pitch.
- Extra per-note sources use their own reduction before final stereo mixing.
- Active production docs and examples lead with WavetableLayer / LayerPad.
- WavetableOsc + VoiceMixer appears as the advanced legacy wavetable path for excluded interaction/feedback behavior, while VoiceMixer remains valid for auxiliary per-voice layers.